Stream Assembly Is an Uncontrolled Treatment in Streaming Intrusion-Detection Benchmarks

ANONYMOUS AUTHOR(S)

Public network captures are rarely usable as evaluation streams as they stand, so streaming intrusion-detection studies assemble them: interleaving capture days, pooling temporally disjoint captures, or replaying records round robin. We show on two benchmarks that this assembly step is not neutral plumbing but an *uncontrolled experimental treatment*. On CICIDS2017, holding the full record multiset identical and changing only the ordering, a fixed positional 70/15/15 chronological split then produces held-out samples that share only 32.5% of their records, at held-out prevalences of 68.235% and 25.2396% — a 42.9954-point difference — and the measured ordering of the two deterministic scorers reverses. Restricting both arms to the 78000 records they both held out removes that reversal: the same scorer leads in both arms there. The reversal is therefore a consequence of which records the assembly hands to the test set, not of the order in which the detector saw its history, and we locate it accordingly. On LITNET-2020, pooling three temporally disjoint captures reports a single 6.4982% operating point that is the equal-weight mean of per-capture held-out prevalences spanning 0.176% to 15.7747%; we present that identity as an audit check rather than a discovery. We also audit the evaluated detector against its description: its reset and growth branches share a predictive term that cancels, so the run-length posterior equals the hazard rate exactly below the run-length cap, while the evaluations spend nearly all their length at or beyond that cap, where the posterior instead wanders; and the score the evaluation consumes is a function of $P(r \leq 5)$, not of $P(r{=}0)$. Every number traces to an archived, hash-verified run manifest, and the sentence-level claim ledger ships with the artifact.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → *Machine learning*; • **General and reference** → *Evaluation*.

Additional Key Words and Phrases: streaming network intrusion detection, benchmark stream construction, evaluation methodology, experimental design, conformal risk control, reproducibility, provenance

1 Introduction

Machine-learning evaluations for network intrusion detection rest on a small number of public benchmarks, and a substantial literature questions what those benchmarks measure [2, 8, 22]. This paper contributes a quantified case study and mechanism audit of one step in that pipeline: the step that assembles capture files into an evaluation stream. CICIDS2017 spans five capture days of very different attack density [12, 23], and LITNET-2020's captures are temporally disjoint [9], so evaluations interleave, pool, or replay records to obtain a single stream with workable splits. We measure what that step does on these two benchmarks.

What this paper claims, and what it does not

Our CICIDS2017 intervention changes the ordering of a fixed record multiset and holds the split *rule* fixed. It does not hold the evaluated *sample* fixed: under a positional split, reordering changes which records land in training, validation and test, and therefore changes held-out membership, held-out prevalence, and within-split order simultaneously. Stream assembly is in this precise sense an **uncontrolled treatment**. We therefore make a pipeline-level claim — assembling the stream differently changes the measured evaluation outcome — and we do *not* claim that order alone causes the change, nor that attack prevalence is excluded as an explanation. Isolating those factors requires a factorial or prevalence-matched design that we identify as the natural next experiment.

Two further scoping statements belong here rather than in a limitations section. First, we make no claim about how often the practice occurs: we have not conducted a systematic literature survey, and the construction we contrast against

is the one used by the audited earlier evaluation of this same work, with interleaving and pooling also discussed in the cited benchmark-criticism literature. Second, the conceptual warning that arbitrary temporal ordering and sampling can distort security-ML evaluation is established prior art [2, 8]; our contribution is the exact same-multiset measurement on a real benchmark, with overlap, relocation, prevalence and outcome reported together, plus the audit of the released detector.

Origin of this version

Earlier versions of this manuscript reported results produced under the pooled and interleaved constructions studied here, and described a scoring rule the released code did not implement. An independent adversarial review and a line-by-line audit of the archived artifacts established both. The present version is the rebuild from that audit: the assembled constructions are retained as explicitly labelled protocols and contrasted against least-transformed alternatives; the detector is described as what the code computes; and every number was regenerated — or, for the retained prevalence sweep, re-derived from the archived pre-audit runs — by a script that wrote, in the same execution, an archived manifest recording the git commit, configuration hash, input SHA-256, seed, environment hash and output path. Section 10 states which measurements this paper shares with those versions and which are new; the per-result-group account and the dated correction history are supplied to the editors confidentially, because printing them here would defeat anonymization.

Contributions

(1) **Stream assembly as an uncontrolled treatment** (Section 5). On the identical 1600000-record multiset (352962 attacks, whole-stream prevalence 22.060125%, permutation verified mechanically), reordering under a fixed positional split yields held-out slices sharing 78000 of 240000 records (32.5%), moves held-out prevalence by 42.9954 points, relocates 103189 attacks out of the held-out slice, and reverses the measured ordering of the two deterministic scorers — a reversal that does *not* survive when both arms are restricted to the records they both held out (Section 6).

(2) **A pooling identity, offered as an audit check** (Section 5.4). LITNET's pooled 6.4982% is the equal-weight mean of the three per-capture held-out prevalences, forced by equal budgets and a divisible round-robin boundary. It is arithmetic, not a discovery; its value is diagnostic — a pooled operating point can correspond to no capture in the pool.

(3) **A scoped method-identity audit** (Section 3). Below the run-length cap the reset and growth branches share a predictive term that cancels, so $P(r_t{=}0)$ equals the hazard rate for any data; at and beyond the cap — where the evaluated runs spend nearly all their length — truncation breaks that identity and the posterior wanders. The score the evaluation consumes is a function of $P(r_t \leq 5)$, not $P(r_t{=}0)$. We state these three facts separately because only the first is an exact result. Measured branch-wise, the auxiliary term is not merely uninformative on the timestamp-ordered held-out slice: it ranks below chance there, and because the deployed score is a maximum it overrides the tail term wherever the tail is small, so the deployed composition ranks worse than its own tail component alone (Section 6.3).

(4) **A provenance discipline that is itself auditable** (Section 9), including the correction history and overlap statement of Section 10.

2 Related Work

Benchmark criticism. Catillo et al. [8] question whether results on public intrusion datasets represent concrete advances and warn specifically that partitioning timestamped records in arbitrary order can alter split representativeness. Arp et al. [2] catalogue sampling bias, mixing incompatible sources and temporal snooping as recurring security-ML pitfalls. Ring et al. [22] survey dataset limitations, and Engelen et al. [12] correct label and flow-construction defects in CICIDS2017; we build on their corrected release. We do not claim to discover that temporal assembly is non-neutral. Our addition is quantitative and specific: the same full multiset under two assemblies, with membership overlap, attack relocation, prevalence and measured outcome reported together on a real benchmark.

Streaming anomaly detection. The baselines evaluated in this rebuild are Half-Space Trees [24] and LODA [21] (streaming), with LOF [6] and ECOD [18] as batch diagnostic references. KitNET [20], xStream [19], RRCF [14], streaming isolation forest [10] and COPOD [17] have implementations in the artifact but no archived evaluation runs in this rebuild, and carry no numbers here. Cao et al. [7] benchmark streaming detectors under controlled anomaly types and drifts and report strong sensitivity to assessment design; our results give one such sensitivity a measured mechanism on an IDS benchmark.

Change-point detection. The evaluated detector descends from Bayesian online change-point detection [1] with a truncated run-length posterior [13, 15]. Section 3 shows the implementation's posterior is data-independent below its cap; van den Burg and Williams [25] document how often change-point evaluations mismeasure their subject.

Conformal and calibrated alerting. Laxhammar and Falkman [16] established conformal anomaly detection for sequential data, with guarantees resting on exchangeability assumptions [26]. Three 2026 lines run concurrent with this work and are distinct from it. FIRCE [**?**] and its successor FADES [4] integrate conformal evaluation into streaming IDS pipelines to drive drift detection and adaptive retraining; both propose detection machinery and evaluate it, whereas this paper proposes no detector and instead audits what an evaluation pipeline measures. Gurjar and Camp [**?**] forecast tail-risk escalation in enterprise alert streams with extreme-value methods over 251 million Suricata alerts — a forecasting task on operational alert volume, not a question about how a benchmark is assembled. None of the three examines the assembly step. Our results are relevant but must be stated carefully: reordering one realized dataset does not establish exchangeability, which is a distributional symmetry. What interleaving can do is *mask temporal dependence, or make exchangeability appear more plausible to a diagnostic* applied to the assembled stream. Conformal methods that relax exchangeability already exist [3].

Cost-sensitive thresholds. The threshold actually implemented is the prior-inclusive Bayes rule of Elkan [11] (Section 3); SRE-style error budgets [5] motivated the original framing.

3 The System Under Study

We characterize the detector as implemented, from measurement. All values here are archived under run `s3_score_threshold_verification`.

3.1 The score the evaluation consumes

The published description defines the anomaly score as the run-length-reset posterior $P(r_t{=}0 \mid x_{1:t})$. The implementation returns

$$s_t \;=\; \max\big(\text{tail}_t,\; 0.25 \cdot P(r_t \le 5 \mid x_{1:t})\big),$$

where tail_t is a χ^2 CDF of the squared standardized residual under a global, slowly adapting diagonal Gaussian. Two facts follow directly. The evaluated score is not the described score; and it is a function of $P(r_t \le 5)$, so results about

$P(r_t{=}0)$ do not transfer to it without further argument. Because $0.25 \cdot P(r \le 5) \le 0.25$, the auxiliary branch can set the score only where the tail term is below 0.25; measured over 49970 post-warm-up records of the timestamp-ordered CICIDS2017 stream it does so on 75.039% of records, at a mean contributed value of 0.002531. A mean magnitude does not establish which branch *discriminates*, since ranking metrics are invariant to the scale of a branch that sets the ranking; we therefore make no claim about that here and measure it directly in Section 6.3.

3.2 The run-length posterior below the cap, and at it

In the implementation the reset and growth branches share the same run-conditional predictive likelihood, which cancels in the posterior normalization. Below the run-length cap, therefore, $P(r_t{=}0)$ equals the hazard rate *h exactly, for any data*. Under a 6σ mean shift with hazard 0.001: mean $P(r{=}0)$ before the shift 0.001; peak in the 50-record window after it 0.001; maximum deviation from the hazard between initialization and truncation 0.

That exact result covers only part of the evaluated regime, and we state the limit plainly. At $t \ge 500$ the run array is truncated before normalization, the cancellation no longer holds, and the posterior wanders (mean 0.01099, maximum 1). Every stream evaluated in this paper is far longer than 500 records, so the published runs spent nearly all of their length in the truncated regime. We have therefore proved data-independence where it is provable and measured behaviour where it is not; we have not established that the auxiliary branch is uninformative over the regime that produced almost all reported scores, and we do not claim it.

3.3 The threshold

The implemented decision threshold is the prior-inclusive Bayes rule [11], $\tau = C_{FP}(1-\rho)/[C_{FP}(1-\rho) + C_{FN}\rho]$, giving 0.655172 at incident prior ρ=0.05 and 0.261745 at ρ=0.22 — not the cost-only $C_{FP}/(C_{FP}{+}C_{FN}) = 0.090909$ described in earlier versions. AUC-PR and AUC-ROC are threshold-free and unaffected; we report no threshold-dependent column, which is also why none of the non-comparable precision/recall/F1 columns appears in this paper.

3.4 Detection delay, not latency

The quantity previously reported as "latency in milliseconds" is a count of records between attack onset and first alert; the evaluation contains no wall-clock instrumentation (0 clock calls in the module). We report it as detection delay in records and report no per-flow compute cost, which this pipeline does not measure.

4 Datasets, Assembly, and Protocol

4.1 CICIDS2017: a timestamp-sorted budgeted subsample

We use the Engelen-corrected release [12]. The evaluation corpus is a **timestamp-sorted budgeted subsample**: 1600000 of 2087997 eligible records, a retention of 76.628%, selected by fixed per-day budgets with proportional stride subsampling. Because the budgets are fixed rather than a common factor, larger days are cut harder and the attack-heavy days are the largest: per-day retention runs from 64.399% (Friday, the densest day) to 93.156% (Tuesday). Measured consequence: the eligible week's prevalence is 24.2065% while the corpus is 22.0601%, a bias of -2.1464 points from the budget step alone. Sorting a stride-subsampled corpus preserves relative order but is not the full event stream; missing flows can break attack runs and alter online sufficient statistics. We measured the prevalence bias and did not measure the effect on attack-family mix, feature distributions or run lengths, so we use "timestamp-sorted budgeted subsample" throughout and avoid claims that require the complete natural chronology. Table 1 summarizes all four streams.

Table 1. The evaluation streams (Stage 1 manifests). Run lengths are contiguous attack runs in the labelled stream; span is the wall-clock extent of the capture.

stream	records	prevalence	held-out prev.	run med/p90/max	span (min)
CICIDS2017 (week)	1600000	22.0601%	68.235%	2/70/2522	6246.71
LITNET udp_flood	500000	14.9384%	15.7747%	1/2/20	3.97
LITNET blaster_worm	500000	0.6562%	3.544%	1/2/5	1.62
LITNET spam	500000	0.0276%	0.176%	1/1/2	39122.23

Table 2. Experimental protocol. Every entry is taken from the code the archived manifests pin, not from intent.

element	setting
features	all numeric columns after dropping the label; non-numeric columns dropped, so no categorical encoding is used
scaling	none applied by the harness
missing / infinite	$\pm\infty$ mapped to NaN, then all NaN filled with 0.0
label rule	y=1 unless the label string is `benign`, `normal`, `0` or `false`
eligibility (CICIDS)	rows whose label contains " - Attempted" dropped
per-day budgets	300k / 300k / 350k / 300k / 350k (Mon–Fri)
subsampling	proportional stride within each class, preserving per-day prevalence
split	positional 70/15/15 over the assembled stream order
detector: hazard	0.001
detector: run-length cap	500
detector: warm-up	30 records
detector: short-run mass	$P(r \leq 5)$, weight 0.25
update timing	score computed from the pre-update predictive; sufficient statistics updated after scoring, on every record including he
label access	the detector and HST see no labels at any point; baselines receive a validation-derived threshold; **ECOD is fitted on b**
metric	`sklearn.metrics.average_precision_score` (AUC-PR) and `roc_auc_score`, scikit-learn 1.8.0, positive class 1, no sa
chance floor	average precision of an uninformative ranking equals the positive-class prevalence p

4.2 LITNET-2020: three disjoint captures

LITNET-2020's captures are temporally disjoint — udp_flood roughly four minutes, blaster_worm under two, spam about four weeks — so no coherent global timestamp order exists across them. We evaluate three per-attack-type streams of 500000 records each, never a manufactured composite; the pooled composite appears only as the explicitly labelled assembled arm of Section 5.4. Multi-window burn-rate alerting over 60/360/4320-minute windows is undefined on captures spanning minutes; we report that as a finding about the benchmark, and no burn-rate result appears in this paper.

4.3 Protocol

Table 2 states the protocol in full so the paper is self-contained; the artifact reproduces it exactly.

Stream assembly, stated as an algorithm:

```
timestamp-order arm:
  rows <- corpus sorted by Timestamp (stable mergesort)
day-round-robin arm:
  groups <- rows grouped by capture date, each in timestamp order
```

Table 3. The assembly contrast. Held-out prevalence, attacks, and the proposed detector's AUC-PR per arm. Every cell is a macro resolving to an archived manifest.

Benchmark	Construction	Held-out prev.	Attacks	AUC-PR (proposed)
CICIDS2017	natural (timestamp)	68.235%	163764	0.728337
CICIDS2017	synthetic (day RR)	25.2396%	60575	0.544998
LITNET udp flood	natural (per type)	15.7747%	11831	0.394428
LITNET blaster worm	natural (per type)	3.544%	2658	0.964091
LITNET spam	natural (per type)	0.176%	132	0.846154
LITNET pooled	synthetic (3-type RR)	6.4982%	14621	0.943056

```
  for k = 0, 1, 2, ...:
      for d in sorted(dates):
          if k < len(groups[d]): emit groups[d][k]
pooled-composite arm (LITNET):
  groups <- per-attack-type streams, each in timestamp order
  emit round robin over sorted(attack_type) as above
split (both arms): i_tr = floor(0.70 n); i_va = i_tr + floor(0.15 n)
```

5 Assembly as a Treatment

5.1 Design and what it identifies

CICIDS2017 admits a clean manipulation of the assembly step: the same record multiset can be presented in timestamp order or in day-of-week round robin. The reordering is verified mechanically to be an exact permutation — both arms hold 1600000 records and 352962 attacks (22.060125%). The assembled arm reproduces the archived earlier evaluation exactly (60575 held-out attacks in 240000 records), so the contrast is against the assembly actually used, not a strawman.

What the design does *not* do is hold the evaluated sample fixed. Under a positional split, reordering changes split membership, so the two arms differ in held-out records, held-out prevalence, training composition and within-split order at once. The estimand is therefore the effect of the complete assembly pipeline under a fixed split rule, and nothing finer. We report the joint change and decline to decompose it.

5.2 What assembly does to the evaluated sample

Held-out prevalence differs by 42.9954 points. The two held-out slices share only 78000 of 240000 records (32.5%); the assembled arm's held-out attacks are a strict subset of the timestamp-ordered arm's, with 103189 attacks relocated into training. The mechanism of the prevalence change is dilution: every held-out attack in the assembled arm is a Friday record (1 of five days contribute any), while Monday–Thursday contribute 162000 attack-free held-out rows. The assembled arm's Friday sub-slice is denser (77.66%) than the timestamp-ordered arm's entire slice: prevalence falls because attack-free days are mixed in, not because attacks are redistributed. This dilution accounts for the prevalence change; it is not by itself an account of the outcome change in Section 5.3.

Table 4. LITNET-2020: AUC-PR per stream against the pooled assembled composite. The detector and ECOD (label-privileged diagnostic reference) are deterministic; HST is a single draw per cell, so no ordering involving HST is asserted. Its measured composite spread across three seeds is 0.23884, 0.177599, 0.367761.

stream	held-out prev.	detector	ECOD	HST (1 seed)
udp_flood (per type)	15.7747%	0.394428	0.321185	0.552675
blaster_worm (per type)	3.544%	0.964091	0.03571	0.147731
spam (per type)	0.176%	0.846154	0.106805	0.008447
pooled (assembled)	6.4982%	0.943056	0.229143	0.23884

5.3 What assembly does to the measured outcome

AUC-PR's chance floor is the positive-class prevalence, which the treatment moves (0.68235 against 0.252396), and additive lift $AP - p$ has a prevalence-dependent ceiling of $1 - p$. We therefore report raw AP, the floor p, and the normalized lift $(AP - p)/(1 - p)$ together, and report no raw cross-arm difference.

- **Timestamp order** ($p = 0.68235$): ECOD $AP = 0.755142$ (normalized lift 0.229158), detector $AP = 0.728337$ (normalized lift 0.144773). HST scores 0.588902 here, 0.093448 below its chance floor, from a single seed, so no placement is asserted for it.
- **Day round robin** ($p = 0.252396$): detector $AP = 0.544998$ (normalized lift 0.391386), ECOD $AP = 0.418966$ (normalized lift 0.222805). HST's three-seed values are 0.513623, 0.427004 and 0.358504 (mean 0.433044, sd 0.077736); its ordering against ECOD flips across those seeds, so it carries no placement.

The measured ordering of the two deterministic scorers reverses between arms, in both raw and normalized terms. **This is a pipeline-level evaluation outcome under the stated protocol, not a statement about method quality.** Three qualifications are load-bearing and are stated here rather than deferred. (i) ECOD is fitted on benign-only training rows and therefore has label access the streaming methods do not; we treat it throughout as a *label-privileged diagnostic reference*, and no superiority is claimed for or against it. (ii) The two arms score different held-out samples, so the reversal is not measured on a common sample. Section 6 measures what happens when that difference is removed, and the reversal does not survive it. (iii) Determinism removes seed variance but not sampling uncertainty from the selected window, correlated attack runs, or the subsample; we report no confidence interval and therefore claim no stable ordering, only a measured one under this protocol.

5.4 LITNET-2020: pooling as an audit check

Round robin *within* a per-attack-type stream is the identity, so the LITNET contrast is compositional: three per-type streams against the pooled composite used by earlier versions. The pooled held-out slice is by construction exactly the union of the three per-type held-out slices — equal budgets, a perfect three-cycle, and a divisible split boundary — so its prevalence 6.4982% is the equal-weight mean of 15.7747%, 3.544% and 0.176%. This is arithmetic, and we present it as an audit check with pedagogical value rather than as a finding: a pooled operating point can correspond to no capture in the pool, and the equal weighting is an artifact of equal budgets. Per-stream values are in Table 4.

6 Isolating What the Treatment Changes

The contrast of Section 5 moves membership, prevalence and order together. Three analyses separate what can be separated from the archived per-record scores of both arms. All three use the metric implementation of Table 2; the

Table 5. Both arms restricted to the 78000 records they both held out. Membership and prevalence are identical by construction; only the history each detector saw before scoring them differs.

arm	detector *AP*	ECOD *AP*	detector AUC-ROC	ECOD AUC-ROC
timestamp order	0.905613	0.844487	0.869219	0.799910
day round robin	0.900371	0.849842	0.864159	0.805657

dumps reproduce the archived arms to 1.793e-05 AUC-PR on the timestamp-ordered arm and exactly on the assembled arm.

6.1 The records both arms held out

The two held-out slices share 78000 records, carrying 60575 attacks at prevalence 0.776603 — they are the Friday records both assemblies happen to place in the test set, which is why their prevalence is far above either arm's. Restricting both arms to exactly those records, and recomputing:

The ordering does not reverse on the shared records. The detector leads ECOD in both arms (margins 0.061125 and 0.050529), and its own score changes by only 0.005242 *AP* between the two histories. This is the round's most consequential result and it constrains the paper's central claim: the reversal reported in Section 5.3 is produced by *which records the assembly places in the test set*, not by the order in which the detector processed its history. We report the reversal as an evaluation-level consequence of assembly and explicitly not as an effect of ordering on detector behaviour.

Two cautions on reading Table 5. The shared records are not a random subsample — they are the intersection two particular assemblies produce, at a prevalence unlike either arm's — so its absolute values characterise that intersection, not the benchmark. And the detector remains prequential, so the two rows still differ in the history preceding each scored record; what is held fixed is the evaluated sample, not the model state.

6.2 Where the split falls

A detector score depends only on the records preceding it, so one pass supports every chronological cut point; ECOD is refit at each cut because its training set moves with the boundary. Over 7 cuts from 60% to 90% of the stream, ECOD exceeds the detector at 3 of them, including the 85% cut this paper's fixed split uses, and the detector leads at the others. The measured ordering is therefore a property of the split point as well as of the assembly; the archived split is not adversarially chosen, but it is not neutral either, and no claim here is a stable property of CICIDS2017. Per-cut values are in the archived findings.

6.3 Which branch discriminates

The score is $\max(\text{tail},\ 0.25 \cdot P(r \le 5))$, and a branch's mean magnitude says nothing about which branch ranks. Scoring the timestamp-ordered held-out slice with each branch alone, using the components `update_score` itself computed:

The tail term alone *outranks* the deployed composition, by 0.103477 *AP* and 0.302658 AUC-ROC. The auxiliary term ranks below chance on this slice (AUC-ROC 0.28189), and because the deployed score is a maximum, that branch overrides the tail wherever the tail is small — which Section 3 measured to be most records. The near-chance AUC-ROC of the deployed detector on this slice (0.526623) is therefore a property of the *composition*, not of either component: a

Table 6. Branch-wise discrimination on the timestamp-ordered held-out slice.

scoring branch	*AP*	AUC-ROC
deployed composition	0.728355	0.526623
tail term only	0.831832	0.829281
auxiliary term only	0.60027	0.28189

Table 7. AUC-PR by prevalence level on the assembled construction (mean over three resampling draws). "Floor" is the achieved held-out prevalence. At the unresampled level all draws see the same records; at resampled levels the spread of every method is the resampling draw. Per-draw values and normalized lift are in the archived findings.

Level	Floor	proposed detector	HST	LODA	ECOD (batch)	LOF (batch)
5%	0.049524	0.367164	0.199261	0.207581	0.166239	0.537596
10%	0.099004	0.447213	0.299702	0.247426	0.252198	0.697003
unresampled	0.252396	0.544998	0.433044	0.341715	0.418966	0.863194
40%	0.399995	0.494217	0.476031	0.399422	0.527364	0.896968
64%	0.639991	0.619546	0.499341	0.524691	0.678404	0.904855

defect in how two signals are combined, not in either signal. We state this as a measured property of one slice of one benchmark, and it supersedes any claim that the tail term simply performs the discriminative work.

7 Prevalence Within One Assembled Construction

An earlier version reported a prevalence sweep as evidence about deployment regimes. Retained here, it is a controlled experiment *on the assembled (interleaved) CICIDS2017 stream* and nothing more: prevalence is varied by stratified resampling at five levels, three draws each. It does not isolate prevalence either — only the unresampled level's draws see identical records, so record identity, repetition and temporal adjacency vary with the nominal level. Table 7 reports *AP* with the achieved held-out prevalence as the floor beside every level.

Two observations, both between the two deterministic cells at the unresampled level, where all draws see identical records. First, the batch label-privileged reference LOF attains $AP = 0.863194$ against the detector's $AP = 0.544998$ on that identical slice. Second, the detector's normalized lift runs 0.33419 at 5%, 0.386472 at 10%, 0.391386 unresampled, 0.157034 at 40%, and -0.056789 at 64% — below zero at the highest level, meaning below what an uninformative ranking achieves. We note that the additive and normalized forms peak at different levels, which is precisely why both are reported. Nothing in this section transfers to the timestamp-ordered stream or to deployment.

8 Repairing the Change-Point Statistic

Section 3 showed the evaluated posterior is pinned below the cap. A standard repair is a prior-predictive term on the reset branch. We implemented **one untuned instance** of that repair — a Normal–Inverse-Gamma prior predictive (Student-t, ν=2, squared scale twice the global variance, standard vague hyperparameters, nothing tuned) — and measured both variants under a 30-minute compute cap whose reductions bind this paragraph: one stream (udp_flood), a 200000-record prefix, one seed. On the synthetic probe the repair responds: $P(r{=}0)$ peaks at 1 after a 6σ shift against 0.001 for the evaluated detector. On real data it ranks near chance: $AP = 0.169912$ against 0.397486 (floor 0.161333; normalized lift 0.010229 against 0.281581), with AUC-ROC 0.50962.

Table 8. One untuned repair saturates the score. Diagnostics over the first 15000 records of the ablation stream; detection metrics on the 200000-record prefix (held-out 30000 records, 4840 attacks, floor 0.161333).

	evaluated detector	one untuned repair
mean posterior mass $P(r{\leq}5)$	0.006472	1
scores exactly at the 0.25 cap	0.004733	0.9274
distinct score values	3899	747
score standard deviation	0.218999	0.159341
AP on the prefix	0.397486	0.169912
normalized lift $(AP - p)/(1 - p)$	0.281581	0.010229
AUC-ROC on the prefix	0.848216	0.50962

Table 8 shows why. Under the repair the posterior sits on short runs at every step (mean $P(r{\leq}5) = 1$), the $0.25 \cdot P(r{\leq}5)$ branch saturates, 0.9274 of scores sit exactly at the 0.25 cap, and 747 distinct values are emitted against 3899. The score is *not* constant — it takes many values and has standard deviation 0.159341 — but it ranks near chance on this sample, which is what an AUC-ROC close to 0.5 indicates. Both variants are thus degenerate in opposite directions: the evaluated one never resets below the cap, this repair almost always resets. We draw no conclusion about what a *tuned* prior-predictive scale would achieve; that requires a training/validation tuning protocol and more than one stream, and selecting it on test labels is forbidden by our protocol.

9 Provenance Discipline

Every number in this paper is a LaTeX macro generated from archived run manifests recording git commit, configuration hash, input SHA-256, seed, environment hash, timestamps and output paths. A gate fails the build if a manuscript number lacks a manifest, if two manifests claim one macro with different values, or if the derived macro index disagrees with the manifests behind it. A claim ledger maps every sentence of the abstract and introduction to its generating run. We are explicit about what this does and does not establish: it provides traceability and integrity — that a displayed number came from a recorded execution on recorded inputs — and it does *not* establish that the code, labels, preprocessing or interpretation are correct. The discipline exists because it caught real defects during this rebuild, including a smoke-test manifest shadowing a full run and a manuscript-bound table cell backed by no manifest; each is a numbered corrected incident in the released artifact.

10 Relationship to Prior Versions of This Work

Earlier versions of this manuscript were publicly posted and are superseded by this one. Because a per-result-group account of what is reused, re-derived, corrected, withdrawn or new would identify those versions — and they are not anonymous — that account is supplied to the editors confidentially rather than printed here, together with the dated version history and the list of claims each correction invalidates.

Three statements can be made in the body without breaking anonymity, and are made here because they bear on how the results should be read. First, some measurements in this paper also appear in the earlier versions: the pooled LITNET composite and the assembled CICIDS arm are the *same measurements*, reported there under a regime interpretation that this paper withdraws and replaces with an assembly interpretation. Second, the timestamp-ordered CICIDS arm, the per-capture LITNET streams, the shared-record analysis, and the method-identity audit have no counterpart in any

earlier version. Third, results on a third dataset that appeared in the earlier versions are withdrawn and are not relied upon anywhere in this paper.

The corrected-incident log in the artifact records each correction with its evidence and closure status. The count of incidents is not itself a quality claim.

11 Threats to Validity

Internal. The assembly treatment is uncontrolled by construction: it moves held-out membership, prevalence and order together, so no single factor is identified by the contrast alone. Section 6 removes the membership difference post hoc and finds the ordering reversal does not survive, which locates the effect in sample membership; it does not decompose the remaining factors, and a factorial design still would. ECOD is a label-privileged diagnostic reference, not a competitor under equal information. Determinism removes seed variance only; we report no confidence interval, and with attack runs of median/p90/max 2/70/2522 an IID row bootstrap would be inappropriate, so an event-block or rolling-origin procedure is the right instrument and is not yet applied. Stochastic baselines appear only with their per-seed values.

External. Two benchmarks, one split rule, one corpus per benchmark. The CICIDS mechanism depends on that dataset's scripted per-day schedule meeting a positional split; it does not establish the direction or magnitude of assembly effects elsewhere. The LITNET identity depends on equal budgets and a divisible boundary. The corpus is a 76.628% budgeted subsample. The timestamp-ordered held-out slice is one 204.2-minute window — what a positional chronological split yields on this corpus, stated rather than engineered around. Timestamp order is the least transformed reference available for CICIDS2017, not a universal gold standard: multi-sensor deployments legitimately merge asynchronous sources, and the durable recommendation is to preserve capture and time metadata, justify the assembly, and report sensitivity to plausible alternatives.

Construct. We do not claim the evaluated detector is change-point detection, we do not claim the untuned repair characterizes corrected change-point detection, and we do not claim that ordering alone causes the outcome change.

12 Conclusion

On these two benchmarks, the step that assembles capture files into an evaluation stream is an uncontrolled experimental treatment: it changes which records are evaluated, at what prevalence, in what order, and thereby what is measured. On CICIDS2017 the same multiset reordered yields held-out slices sharing 32.5% of their records at prevalences 42.9954 points apart, and reverses the measured ordering of the two deterministic scorers — a reversal that disappears once both arms are restricted to the records they share, locating it in the sample the assembly selects rather than in the order it imposes. On LITNET-2020, pooling reports an operating point no capture exhibits. The practical recommendation is not that chronology is always right; it is that assembly belongs in the experimental design and in the provenance record — report capture identity, mixture weights, split membership, held-out overlap, and sensitivity to the plausible alternatives — so that a reported operating point can be recognized as a property of the assembly when it is one.

Data and Artifact Availability

The code, stream-assembly scripts with SHA-256 verification, all run manifests, the macro layer, the claim ledger and the corrected-incident history are archived. During double-anonymous review, the artifact is available through the submission system's anonymous artifact channel.

Acknowledgements: Generative AI Usage

In accordance with the ACM Policy on Authorship, the authors disclose that generative AI tools were used in producing this work. Large language models were used, under continuous human direction, to write and refactor analysis scripts, to draft prose subsequently verified against the archived measurements, and to operate an adversarial review harness that audited the authors' own claims. No generative AI tool is an author of this work, and none produced any reported number: every number was generated by executed code that wrote an archived run manifest in the same execution, and the provenance gate and claim ledger exist in part to make that boundary externally auditable. The authors take full responsibility for the entire content of this work, including any portion drafted with such assistance.

Companion Manuscript Disclosure

A companion manuscript from the same research programme (reference suppressed for double-anonymous review) shares parts of the underlying codebase and predates the audit reported here. The method-identity findings of Section 3 apply to that shared lineage; a correction process for it is in progress and has been disclosed to the relevant editors. Its relationship to the results reported here, and the relationship to the earlier public versions of this manuscript, are stated in Section 10; identifiers are supplied to the editors confidentially.

References

[1] Ryan Prescott Adams and David J. C. MacKay. 2007. Bayesian Online Changepoint Detection. *arXiv preprint arXiv:0710.3742* (2007).

[2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *31st USENIX Security Symposium*. 3971–3988.

[3] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal Prediction Beyond Exchangeability. *The Annals of Statistics* 51, 2 (2023), 816–845. doi:10.1214/23-AOS2276

[4] Seth Barrett, Gokila Dorai, Lin Li, and Swarnamugi Rajaganapathy. 2026. FADES: Adaptive Drift Estimation via Conformal Signals for Streaming Intrusion Detection. *Electronics* 15, 10 (2026), 2114. doi:10.3390/electronics15102114

[5] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering*. O'Reilly Media.

[6] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jorg Sander. 2000. LOF: Identifying Density-Based Local Outliers. In *SIGMOD*.

[7] Yang Cao, Yixiao Ma, Ye Zhu, and Kai Ming Ting. 2024. Revisiting Streaming Anomaly Detection: Benchmark and Evaluation. *Artificial Intelligence Review* 58 (2024). doi:10.1007/s10462-024-10995-w

[8] Marta Catillo, Antonio Pecchia, and Umberto Villano. 2023. Machine Learning on Public Intrusion Datasets: Academic Hype or Concrete Advances in NIDS?. In *53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks – Supplemental Volume (DSN-S)*. 132–136. doi:10.1109/DSN-S58398.2023.00038

[9] Robertas Damasevicius et al. 2020. LITNET-2020: An Annotated Real-World Network Flow Dataset for Network Intrusion Detection. *Electronics* 9, 5 (2020), 800.

[10] Zhiguo Ding and Minrui Fei. 2013. An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window. *IFAC Proceedings Volumes* (2013).

[11] Charles Elkan. 2001. The Foundations of Cost-Sensitive Learning. In *IJCAI*.

[12] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study. In *IEEE Security and Privacy Workshops*.

[13] Paul Fearnhead and Zhen Liu. 2007. On-line inference for multiple changepoint problems. *Journal of the Royal Statistical Society: Series B* (2007).

[14] Sudipto Guha, Nina Mishra, Gourav Roy, and Okke Schrijvers. 2016. Robust Random Cut Forest Based Anomaly Detection on Streams. In *ICML*.

[15] Jeremias Knoblauch and Theodoros Damoulas. 2018. Spatio-temporal Bayesian Online Changepoint Detection with Model Selection. *arXiv preprint* (2018).

[16] Rikard Laxhammar and Göran Falkman. 2014. Online Learning and Sequential Anomaly Detection in Trajectories. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 6 (2014), 1158–1173. doi:10.1109/TPAMI.2013.172

[17] Zheng Li, Yue Zhao, Nicola Botta, et al. 2020. COPOD: Copula-Based Outlier Detection. In *IEEE ICDM*.

[18] Zheng Li, Yue Zhao, Xiyang Hu, et al. 2023. ECOD: Unsupervised Outlier Detection Using Empirical Cumulative Distribution Functions. *IEEE TKDE* (2023).

[19] Emaad Manzoor, Hemank Lamba, and Leman Akoglu. 2018. xStream: Outlier Detection in Feature-Evolving Data Streams. In *KDD*.

[20] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *NDSS*.
[21] Tomas Pevny. 2016. Loda: Lightweight on-line detector of anomalies. *Machine Learning* (2016).
[22] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. A survey of network-based intrusion detection data sets. *Computers & Security* (2019).
[23] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *ICISSP*.
[24] Swee Chuan Tan, Kai Ming Ting, and Fei Tony Liu. 2011. Fast Anomaly Detection for Streaming Data. In *IJCAI*.
[25] Gerrit J. J. van den Burg and Christopher K. I. Williams. 2020. An Evaluation of Change Point Detection Algorithms. *arXiv preprint arXiv:2003.06222* (2020).
[26] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. Algorithmic Learning in a Random World. *Springer* (2005).